



ICBT
CAMPUS



Cardiff
Metropolitan
University

Resilient Multimodal Intelligence

An AI-Driven Lost and Found

Matchmaking System with

Context-Aware Graceful Degradation

CIS6035 Development Project Proposal

Module – CIS6035 – Software Engineering Dissertation Project

Student Name - Dissanayake Thalangama Appuhamilage Dilaksha Haritha Dissanayake

Student ID: GM/BSCSD/06/22 | Cardiff No.: 2036031

Table Of Contents

1. Title	1
2. Introduction	1
3. Current situation leading to problem identification	1
4. Proposed technique to solve the current problem	2
4.1 Methodology	2
4.2 System architecture	3
5. feasibility	6
6. Project Description.....	7
7. Deliverables.....	7
8. Resources Required.....	8
8.1 Hardware Requirements	8
8.2 Software Requirements.....	8
8.3 Third-Party Services	8
9. Expected Output and Outcome	9
9.1 Expected Output	9
9.2 Expected Outcome.....	9
10. Time Plan for Implementation	9
11. Limitation	10
11.1 Production Level Standard Graceful Degradation for Risk Handling.....	10
12.Referances.....	12

1. Title

Resilient Multimodal Intelligence: An AI-Driven Lost and Found Matchmaking System with Context-Aware Graceful Degradation

2. Introduction

Loss of one's belongings is an issue that causes a great deal of distress and loss of productivity. From small stuff such as wallets and keys to costly electronic devices or even identification documents, everyone loses something at some point in life. The number of lost items is incredibly high in places where there are high populations and constant movement like universities, transportation networks, and urban centres, but the methods of retrieving lost items are highly inefficient and scattered.

This project aims at building a cutting-edge, centralized, and extremely intelligent mobile application that acts as a medium connecting the Losers and the Finders, which can help in reducing the loss of important things. This app utilizes innovative technology on the mobile front-end side (Flutter) and on the back-end side uses enterprise-grade technology (Laravel with Python services) and implements an automated matchmaking pipeline for AI.

The system employs computer vision technology to analyse and automatically tag discovered items while matching the geographical coordinates to posts. This significantly minimises the manual labour needed to reunite people with their lost things. The system is also designed on a strong 'privacy by design' principle, which facilitates secure, real-time anonymous interaction to facilitate item exchanges without compromising privacy.

3. Current situation leading to problem identification

At present, the process of recovering the lost items depends heavily on a non-unified, localized, and manual approach.

Non-unified systems: Users must keep track of scattered sources (for example, localized Facebook communities or Twitter/X or even physical Lost and Found bins run independently by different departments). There is no unified source for intelligent matchmaking.

Manual Search: The general search in social media sites means that new and irrelevant postings will overshadow your post quite rapidly. In case, you are the loser of a black leather wallet, then you need to search hundreds of posts, check the poorly taken photos, and decipher varied textual description.

"Connection Barrier" (Inaccessibility): When users put up any notification for a lost item through instant messaging services such as WhatsApp, Telegram, etc., the scope of audience for that notification is restricted only to the users' personal contact list or localised groups. If a person finding the item does not belong to the user's personal contact list or network, he would

never come across the notification. You can understand the point here; there is no world network reach. The possibility of retrieving lost items becomes highly unlikely.

Privacy and Security Issues: Sensitive personal data, including one's phone number, address, or email ID, becomes a necessity while putting up any public notice regarding lost items on social media. It makes the users highly susceptible to harassment, intimidation, or even fraud attempts like phishing and scamming.

Inadequacy of Verifiability and Protection Against Fraud: No such procedure exists to verify any of the above methods. The finder-loser exchanges are based entirely on faith and trust, creating room for unscrupulous elements to claim items not belonging to them.

4. Proposed technique to solve the current problem

To effectively mitigate the identified inefficiencies in lost item recovery, this project proposes a high-availability, multi-tiered software ecosystem. The architecture is engineered to shift from the current manual, fragmented approach to an automated, AI-driven matchmaking paradigm that emphasizes privacy, scalability, and resilience.

4.1 Methodology

The engineering lifecycle for this project follows an iterative Agile Scrum framework mapped across a strict twelve-week timeline. This approach allows for continuous delivery, adaptable risk management, and regular evaluation of system increments. By breaking the development down into targeted two-week sprints, the core infrastructure, database models, and communication layers can be thoroughly stabilized and tested before integrating the more complex multimodal artificial intelligence components.

4.1.1 Mobile Client Architecture: Clean and Modular Design

Rather than relying on basic presentation patterns that tend to tangle code, the Flutter mobile application is built upon a strict Clean and Modular Architecture. This design clearly separates the user interface from the underlying business logic and external data consumption layers.

By isolating individual features like user authentication, report submission, and local coordinate masking into distinct modules, the codebase remains highly organized and easy to maintain. For a final year dissertation, this approach demonstrates advanced architectural competence because it prevents unintended dependencies, ensures that individual components can be tested in isolation, and allows the mobile interface to scale smoothly.

4.1.2 Decoupled Multi-Tier Backend Strategy

To ensure high availability, robust security, and a clear separation of concerns, the backend infrastructure avoids a monolithic structure. Instead, it utilizes a decoupled multi-tier service layout that separates standard application routing from heavy intelligence workloads.

The Core API Gateway and Orchestration Layer is built with PHP and the Laravel framework. This layer acts as the primary traffic controller. It handles core requests from the mobile client, manages secure token-based authentication via Laravel Sanctum, executes spatial transactions within the PostgreSQL database, and orchestrates real time bidirectional chat sessions using Laravel Reverb.

Because artificial intelligence processing requires significant computational power, all intelligence tasks are offloaded to a dedicated Python microservice built with FastAPI. This isolated microservice drives the core innovation of the platform by executing a dual pipeline similarity engine. First, it uses a Large Language Model to process the textual descriptions, tags, and contextual data submitted by users, computing semantic text similarity scores between lost and found reports. Second, uploaded images are analysed concurrently using computer vision to extract key features, colors, and object classifications, comparing them against existing database entries to calculate visual similarity.

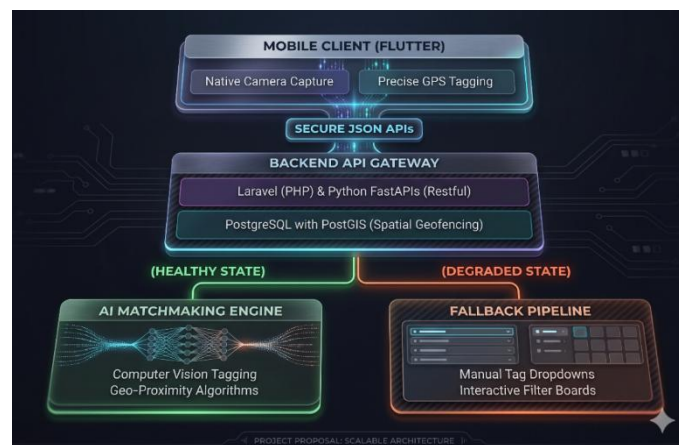
By combining semantic text correlation with image feature matching, the system generates a highly accurate multimodal confidence score. The primary Laravel gateway communicates with this Python service over secure low latency REST APIs, ensuring that heavy inference tasks never block standard routing or slow down the mobile app.

4.1.3 DevOps and Continuous Integration Pipelines

To maintain rigorous quality control and streamline integration throughout the sprints, the project actively incorporates standard DevOps practices. All source code is version controlled via Git and maintained in a clean GitHub repository.

Automated Continuous Integration and Deployment pipelines are configured using GitHub Actions. Every time new code is committed, the pipeline automatically triggers a comprehensive suite of tests covering isolated unit tests for processing logic, integration tests for database geofencing queries, and API endpoint validations. This guarantees that the matchmaking algorithms and graceful degradation fallback mechanisms remain completely functional as the system evolves.

4.2 System architecture



The overall system architecture serves as a dependable foundation that links the mobile interface directly to our specialized backend services. When designing this layout, the main priority was to ensure the platform remains stable and accessible even if external services fail unexpectedly.

Mobile Client Layer

We built the front end using the Flutter SDK so that a single codebase runs natively on both Android and iOS devices. This approach lets the application tap straight into native hardware features, making it easy to capture clear photos and pull accurate GPS coordinates without unnecessary lag.

API Gateway and Persistence Layer

At the core of the backend sits a Laravel gateway that handles incoming traffic and directs it to a PostgreSQL database. By adding the PostGIS spatial extension to this database, the system can run complex geographic queries and set up precise geofencing boundaries extremely fast.

Dual State Execution Routing for Risk Handling

To keep the platform running smoothly during unexpected service outages, the backend uses a dynamic routing design. This setup constantly checks system health and decides how to process incoming data on the fly.

Healthy State Execution

Under normal conditions, when all external connections are working perfectly, the system sends uploaded images, text descriptions, and fuzzed coordinates straight to the artificial intelligence engine. A dedicated Python microservice takes over from here to run a synchronized multimodal analysis. It passes the text descriptions through a Large Language Model to check for semantic meaning while simultaneously using computer vision algorithms to identify visual patterns in the images. The platform then combines these results with our spatial distance calculations to give the user a highly accurate match confidence score.

Degraded State Execution via Graceful Degradation Fall-back

External artificial intelligence services can sometimes slow down or drop offline entirely. If that happens, the backend catches the errors automatically, so the application does not freeze or crash. Instead, it smoothly shifts the user over to a secure fall-back pipeline. This backup workflow lets users find items manually by giving them clear category drop down menus and interactive search filters, ensuring the core service remains fully usable no matter what.

4.3 Artificial Intelligence and Machine Learning Techniques

The intelligence behind the matchmaking pipeline depends on the synchronized execution of three core algorithmic processes designed to correlate lost and found data accurately.

4.3.1 Multimodal Similarity Engine: Computer Vision and Large Language Models

When a user submits a report, the isolated Python microservice processes the payload through a dual pipeline similarity engine to extract structured features and compute semantic alignment.

To maintain strict risk handling, control operational costs, and maximize classification accuracy within the twelve-week development scope, the visual inference is tightly constrained to three target domain categories. These categories are consumer electronics including laptops and mobile phones, documents and money holders like identification cards and wallets, and luggage such as suitcases and backpacks.

The engine processes incoming data streams concurrently using two distinct methods. First, uploaded images are routed to a computer vision model to perform automated feature extraction, identifying broad categories, specific subcategories, and dominant colors. Second, the accompanying textual descriptions and contextual notes are parsed by a Large Language Model. The system computes a semantic text similarity score while matching visual feature vectors against existing database entries. Combining these visual and text metrics yields a highly accurate normalized multimodal confidence score.

4.3.2 Geospatial Proximity Algorithms

Simultaneously, the spatial engine queries the PostGIS database to evaluate distance metrics between the reported point of loss and the point of recovery. The algorithm calculates distance radii to determine proximity. If a newly found item registers within a predefined logical geofence relative to a lost item report, the system applies a mathematical boost to the overall match confidence score. This ensures that geographically relevant results are prioritized automatically.

4.3.3 Automated Verification Matchmaking

When a high confidence match is established through combined multimodal tagging and proximity correlations, the system initializes an automated ownership authentication protocol. To prevent fraudulent claims, the artificial intelligence pipeline generates contextual verification queries regarding unexposed item details. The claimant must successfully answer these dynamic questions to prove ownership before the platform dynamically provisions a secure anonymous WebSocket chat room for the final exchange.

4.4 Technologies & Tools

The platform relies on a highly robust stack of open-source and enterprise-grade tools, mapped specifically to their architectural layers:

Architectural Layer	Proposed Technology / Tool	Core Function & Justification
Mobile Frontend	Dart & Flutter SDK	Delivers a highly responsive, cross-platform client (iOS/Android) from a single codebase utilizing Clean Architecture.
State Management	Riverpod (Flutter Riverpod)	Provides scalable, reactive, and maintainable application state management with dependency injection and efficient UI updates aligned with Clean Architecture principles.

Core Backend API	PHP 8.2 & Laravel 11	Orchestrates main routing, business logic, secure API endpoints, and database interactions.
Authentication	Laravel Sanctum	Enforces robust, token-based security for all RESTful API interactions.
Real-Time Relay	Laravel Reverb	Powers real-time, low-latency bidirectional WebSocket infrastructure for anonymous chat lobbies.
AI Microservice	Python 3 (FastAPI / Flask)	Acts as an isolated, lightweight microservice handling computational AI orchestration.
Computer Vision	Google Cloud Vision API	Performs advanced feature extraction, automated object detection, and metadata tagging.
Database Engine	PostgreSQL with PostGIS	Serves as an enterprise relational database; PostGIS handles complex spatial geofencing calculations natively.
Mapping Services	Google Maps Platform API	Facilitates client-side coordinate mapping, spatial conversions, and geographic location processing.
DevOps & IDEs	Git, GitHub, VS Code, Android Studio, Postman	Supports strict version control, automated CI/CD pipeline integration, and API endpoint documentation.

5. feasibility

Technical Feasibility -: Flutter and Laravel are already well-established and extremely stable with amazing documentation. With built-in support for WebSocket connections (Reverb) and authentication for APIs with tokens (Sanctum), the implementation of real-time communication with the use of the Laravel framework reduces difficulty and risk of any sort of vulnerabilities.

Operational Feasibility: The search for lost items is a basic human need that applies universally to all people. It requires almost no investment to begin and has very quick adoption due to reducing the tedious process of looking and posting publicly on social media sites.

Economic Feasibility: Open-source, free technologies (Flutter, Laravel, PostgreSQL) are used in the development of the main codebase. AI activities are not entirely free, though. A production-grade solution will incur usage-based expenses, even though Google Cloud Vision

and Google Maps APIs offer large free developer levels that easily manage prototyping and academic testing. The system's "Graceful Degradation" pipeline and strict architectural scope limit, which limits processing to three target categories, serve as built-in cost-control mechanisms to reduce this risk by methodically cutting down on pointless API calls and maintaining runtime costs within a highly predictable, cost-effective envelope.

Legal & Ethical Feasibility: The application is built using a "privacy-by-design" architecture. Exact GPS locations are masked on the client-side using coordinate fuzzing, and absolute anonymity is preserved via WebSocket relays, making the platform fully compliant with modern data protection standards (such as GDPR). A formal academic ethics approval application will be submitted and cleared before any public testing or data collection.

6. Project Description

This project will produce a fully functional, cross-platform mobile application ecosystem for managing lost and found items using AI-driven automation. The system consists of three primary components: (1) a Flutter-based mobile client for both iOS and Android used by Reporters and Finders; (2) a Laravel/Python RESTful backend API connected to a PostgreSQL spatial database responsible for AI orchestration, geospatial matching, and secure communications; and (3) a web-based administration and moderation dashboard for institutional oversight.

The project will demonstrate advanced software engineering competencies including AI/ML integration via third-party APIs, real-time bidirectional communication (WebSocket), spatial database querying (PostGIS), fault-tolerant system design (graceful degradation), and comprehensive software testing methodology (unit, integration, and user acceptance testing). The project directly addresses a real-world problem, follows an Agile development methodology, and is aligned with industry-standard best practices for secure, privacy-compliant application development.

7. Deliverables

Administrative Moderation Dashboard: A web-based control panel built with Laravel that allows system administrators to review user reports, monitor system health, and moderate flagged postings.

Entity-Relationship Diagrams (ERD), use-case diagrams, sequence diagrams, and full automated test suites (unit, integration, and API testing) are all part of this detailed software engineering dissertation report.

Approved Ethics Application: A signed, completed ethics compliance agreement that addresses data privacy and geographical location monitoring (included as Appendix 2).

Executable Client Application: A cross-platform mobile application (Android .apk and iOS simulator build) developed in Flutter using Clean and Modular architecture.

Production-Ready Backend: A fully functional, 3-tier REST API developed in Laravel/Python, integrated with a PostgreSQL/PostGIS database and an Object Detection AI LLM.

DevOps Pipeline Repository: A version-controlled GitHub repository demonstrating active CI/CD pipelines for automated testing and deployment.

Comprehensive Engineering Documentation: A complete dissertation report including UML diagrams (Use Case, Sequence, Class diagrams), system architecture blueprints, and API documentation (e.g., Swagger/Postman collections).

Software Testing Suite: Documented evidence of unit, integration, and user acceptance testing to validate the matchmaking algorithm and graceful degradation fallback.

8. Resources Required

8.1 Hardware Requirements

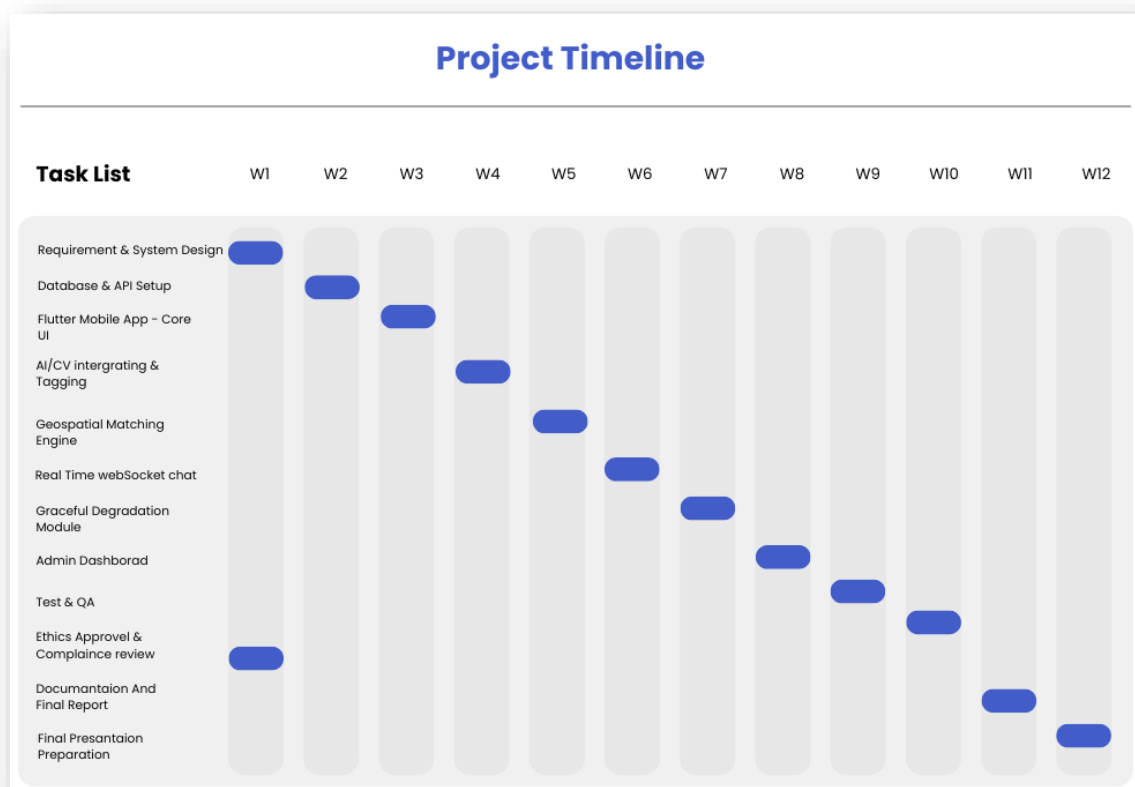
- For operating emulators and servers at the same time, the development computer (either PC or laptop) should have at least 16 GB of RAM.
- Physical Android device (testing hardware like GPS & camera).
- Xcode Simulator or an iOS device (test for cross-platform compatibility).

8.2 Software Requirements

- Dart Programming Language and Flutter SDK (cross-platform mobile development).
- PHP 8.2 with Laravel 11 framework (WebSocket server and backend API).
- Python 3 with Flask or FastAPI (microservice layer for AI).
- PostgreSQL Geographic Relational Database Management System with PostGIS extension.
- Android Studio and Visual Studio Code (main integrated development environment).
- Postman (API endpoints testing and documentation tool).
- GitHub and Git (version control and project management platform).

8.3 Third-Party Services

- Google Maps Platform API: to manage the processing of geographical locations, coordinate conversion, and mapping.
- Google Cloud Vision API (or any other computer vision service): to perform automatic feature extraction and analysis of images using artificial intelligence.
- Laravel Verbs: for connecting to WebSocket servers.



11. Limitation

1 Budget and Financial Constraints

Using external artificial intelligence and mapping interfaces creates usage-based expenses for a production grade solution. While free developer tiers handle prototyping, running a full-scale platform requires strict financial management. To control risks, the graceful degradation fallback pipeline and category scope limits act as built in cost control mechanisms. This approach cuts down on pointless API calls to keep runtime expenses within a predictable, cost effective budget envelope.

2 Operational Scope Constraints

To maintain high matching accuracy, reduce request latency, and ensure completion within the strict twelve-week deadline, the operational scope is restricted to specific categories. These classes are limited exclusively to consumer electronics, documents and money holders, and luggage and bags.

3 Artificial Intelligence Granularity Limits

While computer vision technology detects broad external features rapidly, it cannot reliably differentiate between two related items of the same category. Identical items lacking distinct physical markers cannot be separated automatically by the intelligence models.

4 Technical and Hardware Dependencies

Platform functionality relies entirely on the user having access to a contemporary smartphone. The device must be capable of connecting to the internet, taking clear pictures, and tracking locations via active GPS hardware.

5 Human Element and Manual Engagement

The physical return of recovered property ultimately depends on the honesty of the finder, despite advancements ensuring secure anonymous connections. Furthermore, when automated intelligence processes break down, successful recovery depends entirely on the willingness of the user to engage actively with manual selection and verification steps.

11.1 Production Level Standard Graceful Degradation for Risk Handling

Enterprise grade software systems must account for external third-party vulnerabilities and unexpected downtime. In this proposal, graceful degradation serves as a core production level standard engineered specifically for operational risk handling, technical fault tolerance, and strict cost limitation.

If the primary computer vision service encounters unexpected downtime, exceeds predefined budget thresholds, or times out, the backend application gracefully catches the resulting errors. Instead of crashing or displaying fatal exceptions to the end user, the ecosystem seamlessly transitions into a structured manual fallback workflow.

Finder Side Risk Mitigation

When automated classification fails or responds too slowly, the mobile interface dynamically renders a hierarchical drop-down menu. This prompts the finder to manually input the classification tags of the item, moving smoothly through categories like Electronics to Smartphone and selecting the dominant color Black.

Loser Side Matchmaking Failover

If automated match confidence scores cannot be computed because artificial intelligence tags are unavailable, the interface for the loser smoothly shifts from automated matchmaking notifications directly to an interactive search and filter dashboard. The system prompts the user by asking if they would like to view potential manual matches discovered in the database. Upon confirmation, the user is presented with these potential matches to review and verify autonomously.

This degradation mechanism acts as a critical cost control framework. It systematically halts pointless and failing API requests while ensuring the platform maintains high system availability and a smooth user experience under all operational conditions.

12. References

- Al-Turki, M. and Doheir, M. (2020) 'A Framework for Automated Lost and Found Systems Using Machine Learning', *International Journal of Advanced Computer Science and Applications*, 11(8), pp. 45–53.
- British Lost Property Office (2022) *Annual Lost Property Statistics Report*. London: HMSO.
- European Parliament (2016) *General Data Protection Regulation (GDPR) — Regulation (EU) 2016/679*. Official Journal of the European Union.
- Fowler, M. (2018) *Refactoring: Improving the Design of Existing Code*. 2nd edn. Addison-Wesley Professional.
- Google (2024) *Flutter Documentation*. Available at: <https://docs.flutter.dev> (Accessed: May 2026).
- Google Cloud (2024) *Cloud Vision API Documentation*. Mountain View: Google LLC. Available at: <https://cloud.google.com/vision/docs> (Accessed: May 2026).
- Khan, F., Rehman, A. and Osman, M. (2021) 'Ownership Verification in Digital Property Recovery Systems', *IEEE Access*, 9, pp. 88741–88753.
- Laravel LLC (2024) *Laravel 11 Documentation*. Available at: <https://laravel.com/docs> (Accessed: May 2026).
- Napolitano, M., DeLong, A. and Bhatt, P. (2019) 'Digital Lost and Found: Evaluating Online Platforms for Item Recovery', *Journal of Information Systems*, 34(2), pp. 112–129.
- Nygard, M. T. (2018) *Release It! Design and Deploy Production-Ready Software*. 2nd edn. Pragmatic Bookshelf.
- Sommerville, I. (2016) *Software Engineering*. 10th edn. Pearson Education.
- Worsley, S. (2021) 'Privacy-by-Design in Mobile Application Development: A Systematic Review', *Computers & Security*, 102, p. 102152.